Sprint 8 · Resultados de los cuadernos (sin código)

IFCD107 · Especialista en Inteligencia Artificial

- **Para explicar a clase** qué pide cada ejercicio y qué se obtiene, sin enseñar código.
- **Clase** = ejercicio del cuaderno «en clase» · **Casa** = solo en el cuaderno completo (`para_casa/`).
- Los números son los que salen al ejecutar los cuadernos con los datos del curso (turisdata).

S08_01 · Árboles de decisión

Ej.	Qué pide	Qué obtenemos
1 · clase	Con el árbol pequeño: ¿por qué variable empieza preguntando y cuántas hojas tiene?	Empieza por la **antelación de la reserva**; tiene **8 hojas**
2 · clase	Árbol sin límites: AUC en entrenamiento y en prueba	Entrenamiento **1.000** pero prueba **0.587**, con **1 206 hojas**: se ha aprendido los datos de memoria
3 · clase	Podar: elegir el mínimo de reservas por hoja con validación cruzada	Mejor valor: **80 reservas por hoja** → AUC en prueba **0.716** (frente a 0.587) con **46 hojas**
4 · casa	Las 3 variables más importantes del árbol podado	**Antelación, sin depósito y canal directo**: suman el **70 %** de la importancia

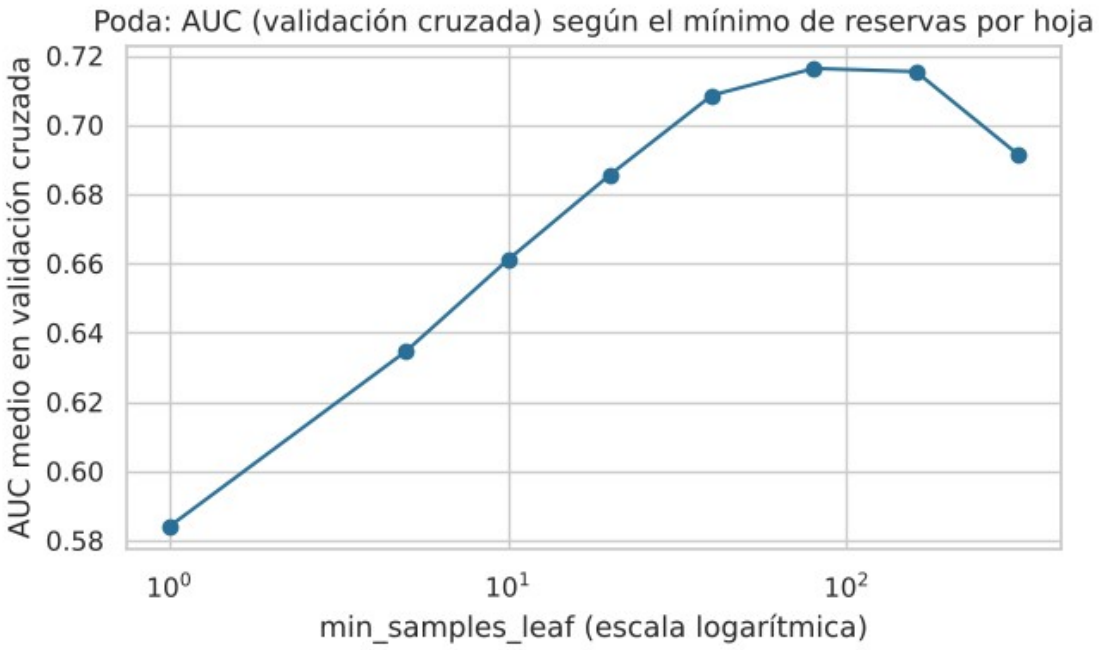


Figura 1. *Ejercicio 3 · AUC según el mínimo de reservas por hoja: sube al podar y cae si se poda demasiado; el máximo está en 80.*

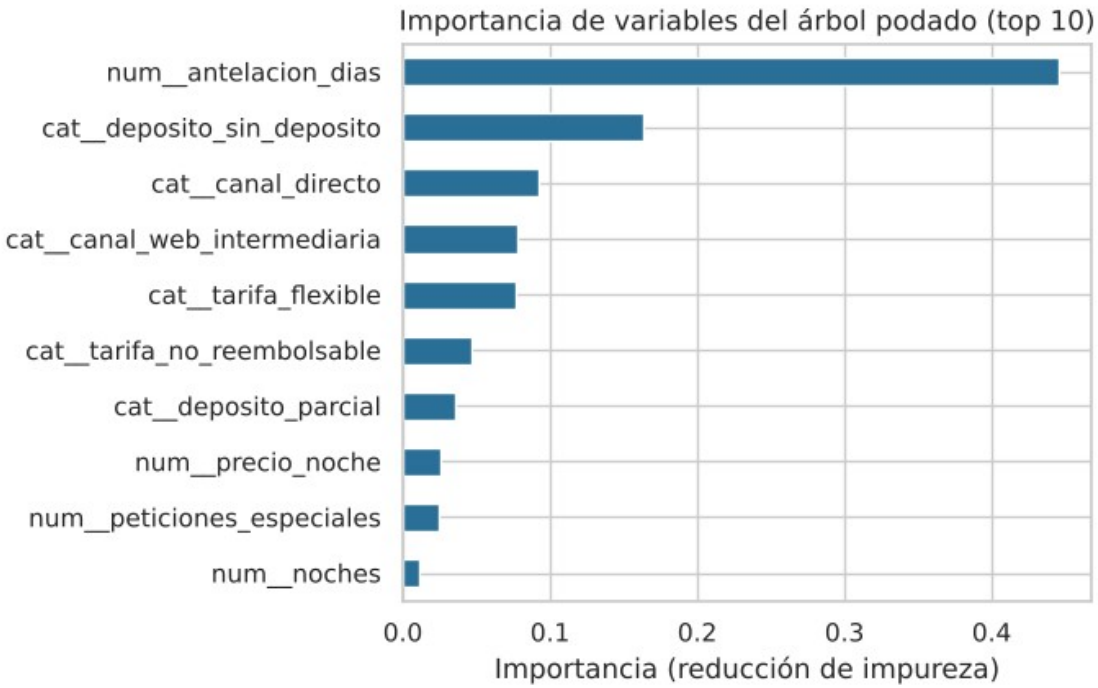


Figura 2. *Ejercicio 4 · Importancia de las variables del árbol podado: la antelación domina.*

S08_02 · Random Forest

Ej.	Qué pide	Qué obtenemos
1 · clase	Entrenar un bosque de 300 árboles y compararlo con un árbol podado	Bosque **0.734** de AUC frente a **0.716** del árbol
2 · clase	Cuánto mejora al añadir árboles	De 5 a 400 árboles: **+0.103**; de 100 a 400: solo **+0.005** (rendimientos decrecientes)
3 · casa	Variable más importante por permutación y cuántas son casi irrelevantes	La más importante es la **antelación**; **6 variables** casi no aportan

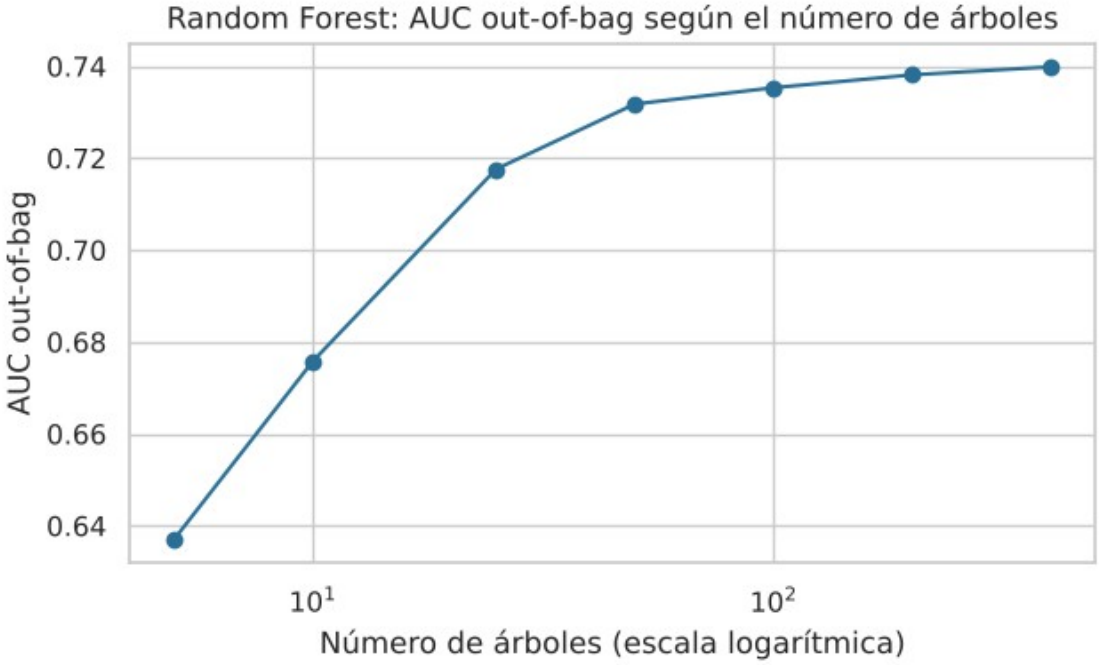


Figura 3. *Ejercicio 2 · AUC out-of-bag según el número de árboles (escala logarítmica): sube rápido y se aplana.*

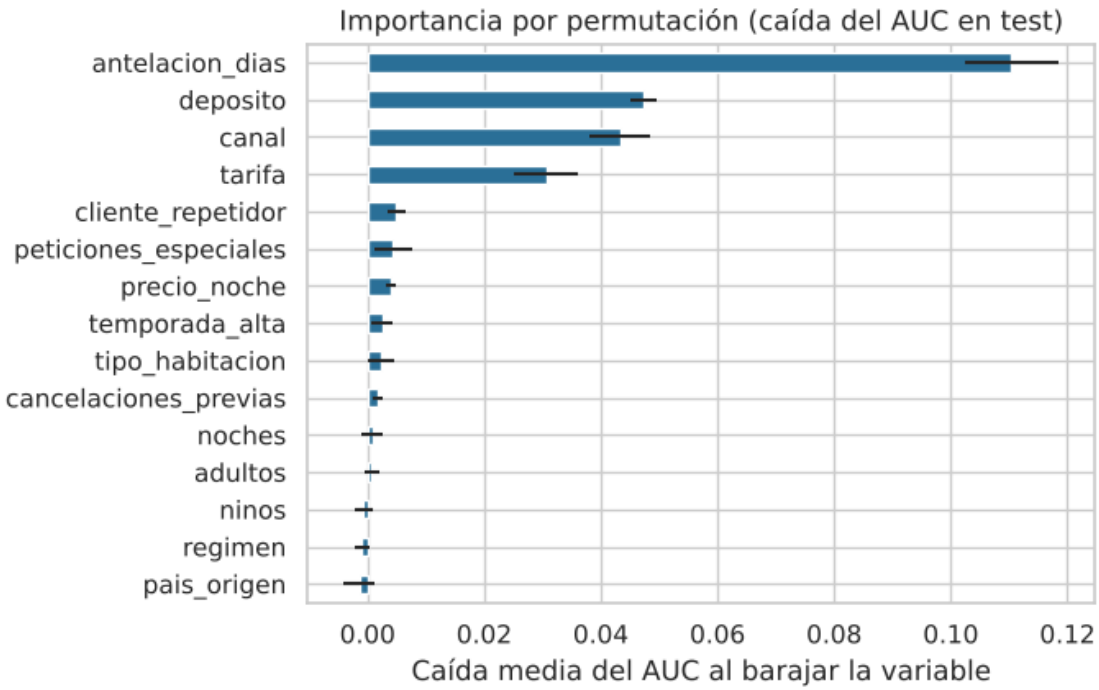


Figura 4. *Ejercicio 3 · Importancia por permutación: cuánto cae el AUC al barajar cada variable. Antelación, depósito, canal y tarifa son las que importan.*

S08_03 · Boosting: XGBoost y LightGBM

Ej.	Qué pide	Qué obtenemos
1 · clase	Qué número de árboles es el mejor en validación	El mejor es **12 árboles**; seguir hasta 300 pierde **0.039** de AUC (sobreajuste)
2 · clase	XGBoost con parada temprana	Para solo en **69 árboles** · AUC en prueba **0.745**
3 · casa	LightGBM con parada temprana	Para en **49 árboles** · AUC en prueba **0.745**: igual que XGBoost

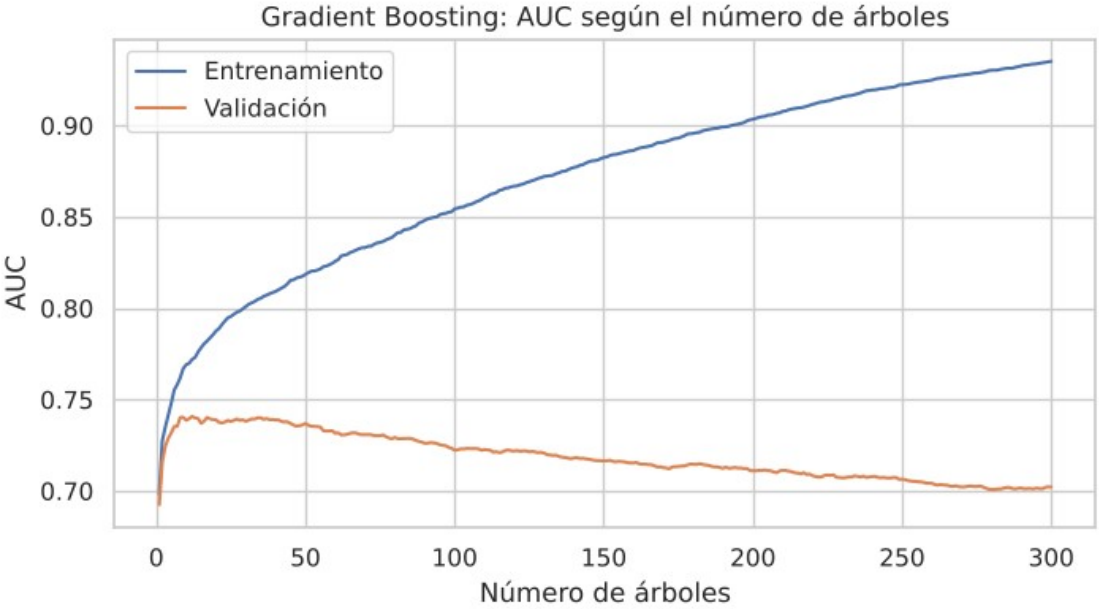


Figura 5. *Sección 2 · Con más árboles el AUC de entrenamiento sube sin parar (azul) pero el de validación baja (naranja): hay que parar a tiempo.*

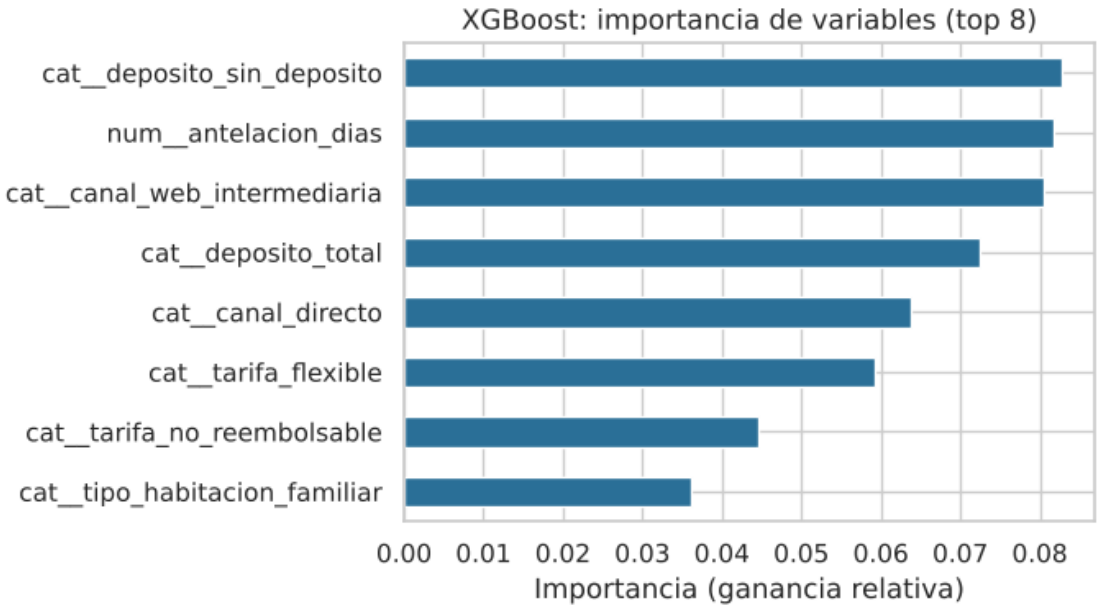


Figura 6. *Sección 5 · Qué mira el modelo: depósito, antelación y canal de intermediarios encabezan la lista.*

S08_04 · Máquinas de vectores de soporte (SVM)

Ej.	Qué pide	Qué obtenemos
1 · casa	SVM lineal frente a SVM con kernel RBF en datos curvos	Lineal **80.8 %** · RBF **90.8 %** con **91 vectores de soporte**
2 · casa	Con y sin escalar las variables	AUC **0.735** escalando frente a **0.671** sin escalar
3 · casa	Buscar los mejores C y gamma con validación cruzada	Mejor: **C = 10, gamma = 0.003** → AUC **0.752**
4 · casa	Cuánto tarda al cuadruplicar las filas	4 veces más filas → **16.9 veces** más tiempo (0.05 s → 0.84 s)

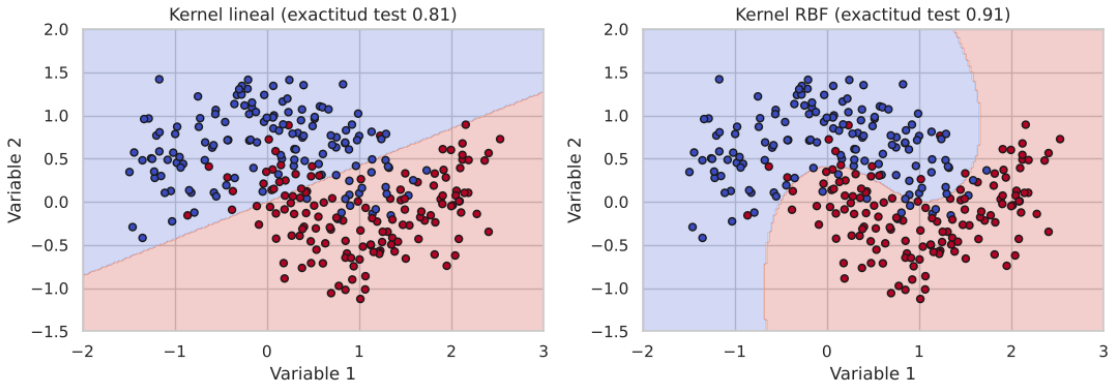


Figura 7. *Ejercicio 1 · Frontera de decisión: la lineal (izquierda) corta recto; la RBF (derecha) se curva y se adapta.*

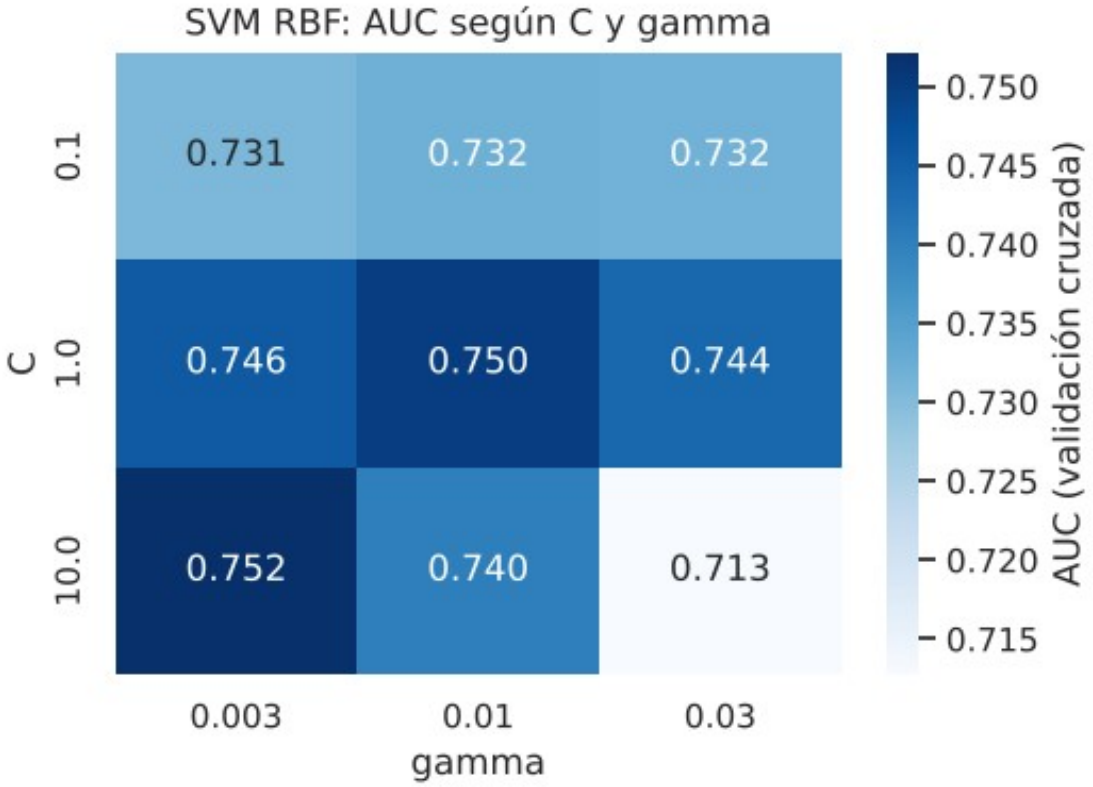


Figura 8. *Ejercicio 3 · AUC para cada pareja C–gamma: el mejor es C = 10 con gamma = 0.003 (0.752).*

S08_05 · Búsqueda de hiperparámetros

Ej.	Qué pide	Qué obtenemos
1 · casa	Búsqueda en rejilla para un Random Forest	**8 combinaciones** (24 entrenamientos en 18 s) · mejor AUC **0.747**
2 · casa	Búsqueda aleatoria para LightGBM (25 combinaciones)	Mejor AUC **0.749**, tarda unos 5 minutos
3 · casa	¿Hay un ganador claro entre las 25 configuraciones?	**13 de 25** quedan a menos de un error estándar de la mejor; la más sencilla usa solo **4 hojas**
4 · casa	Comprobar los dos mejores modelos con los datos de prueba	Rejilla + RF: **0.747** → **0.734** · aleatorio + LightGBM: **0.749** → **0.740** (la prueba siempre sale algo peor)

S08_06 · Reto: tabla de clasificación de modelos

Ej.	Qué pide	Qué obtenemos
1 · reto	Registrar un modelo de referencia y una regresión logística	La logística llega a **0.755** de AUC (el de referencia, 0.50) y es explicable (nota 4)
2 · reto	Registrar otros cuatro modelos (árbol, bosque, LightGBM, SVM)	Tabla con **6 modelos**: rendimiento, tiempo, tamaño y explicabilidad
3 · reto	Elegir pesos según lo que valora el cliente (rendimiento, explicabilidad, coste)	Con pesos **0.5 / 0.3 / 0.2** el campeón es la **regresión logística**
4 · reto	Medir al campeón una sola vez con los datos de prueba	AUC en prueba **0.762** (en validación, 0.755)

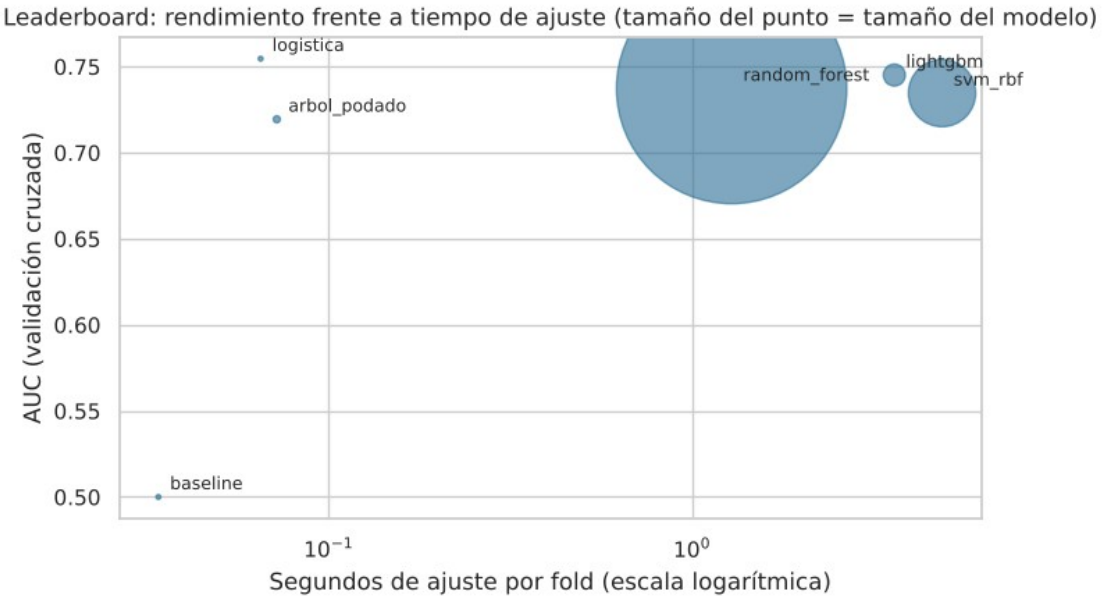


Figura 9. *Reto · Rendimiento frente a coste de ajuste: el tamaño de la burbuja es el tamaño del modelo. La logística es la más rápida y la mejor.*

La historia del sprint

- **1 206 → 46** · Podar el árbol lo hace **más pequeño y mejor**: AUC en prueba de 0.587 a 0.716.
- **0.745** · **Boosting** (XGBoost y LightGBM) con parada temprana llega a 0.745 con muy pocos árboles.
- **0.75** · Da igual cómo se busque: rejilla o aleatoria, el AUC en validación cruzada se queda **en torno a 0.75**.
- **Logística** · En el reto, la **regresión logística** gana: casi el mejor AUC (0.762) y es la más explicable.